

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Короткевич

J3212

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Цель работы

Изучить принципы взаимодействия веб-приложения с внешними API, а также реализовать авторизацию пользователей с помощью мокового API

Задача

- Подключить разработанное в ЛР1 веб-приложение к внешнему API
- реализовать взаимодействие с сервером с помощью библиотеки Axios
- создать моковое API с использованием JSON Server
- реализовать регистрацию и авторизацию пользователей;
- реализовать динамическую загрузку данных

Ход работы

1. Создание мокового API

Для имитации серверной части был использован инструмент JSON Server совместно с json-server-auth.

Был создан файл db.json, содержащий данные пользователей, моделей и других сущностей.

Пример структуры:

```
{
  "users": [
    {
      "id": "u1",
      "email": "nastya@gmail.com",
      "password": "$2a$10$Iqz/ES3IHJ$Sa6FgNTA/qR.za.wE8o5T75q/J0fWgE0fWAoVPifsxK",
      "name": "Anastasia",
      "role": "ML Engineer",
      "description": "Разрабатываю NLP и CV модели, публикую датасеты и эксперименты.",
      "avatar": "https://i.pravatar.cc/150?img=5"
    },
    -
  ]
}
```

```

"models": [
  {
    "id": "m1",
    "type": "model",
    "title": "BERT Sentiment RU",
    "description": "Модель для анализа тональности русскоязычных отзывов и комментариев",
    "author": "Anastasia",
    "authorId": "u1",
    "task": "NLP",
    "license": "MIT",
    "framework": "PyTorch",
    "size": "420 MB",
    "sizeCategory": "medium",
    "version": "1.2.0",
    "downloads": 1840,
    "stars": 312,
    "forks": 48,
    "metrics": {
      "Accuracy": "94.2%",
      "F1-score": "91.8%",
      "Precision": "92.5%",
      "Recall": "91.1%"
    },
    "usageExample": "from transformers import pipeline\nnclassifier = pipeline('text-classification', model='bert-sentiment-ru')\nresult = classifier('Крыс очень понрави",
    "reproducibility": {
      "datasetVersion": "ru_reviews_v2",
      "trainingConfig": "batch_size=32, epochs=5, lr=2e-5",
      "randomSeed": "42",
      "hardware": "NVIDIA Tesla T4",
      "logs": "available"
    }
  },
],

```

```

"datasets": [
  {
    "id": "d1",
    "type": "dataset",
    "title": "RU Reviews 500K",
    "description": "Набор русскоязычных отзывов с разметкой тональности для NLP-задач",
    "author": "Anastasia",
    "authorId": "u1",
    "task": "NLP",
    "license": "CC BY 4.0",
    "framework": "CSV / JSON",
    "size": "1.4 GB",
    "sizeCategory": "large",
    "version": "2.1.0",
    "downloads": 2210,
    "stars": 411,
    "forks": 80,
    "metrics": {
      "Records": "500 000",
      "Classes": "3",
      "Language": "Russian"
    },
    "usageExample": "import pandas as pd\nndf = pd.read_csv('ru_reviews.csv')\nprint(df.head())",
    "reproducibility": {
      "collectionProtocol": "reviews scraping + manual labeling",
      "annotationAgreement": "0.87 Cohen's kappa",
      "splits": "train/valid/test = 80/10/10",
      "logs": "available"
    }
  },
],

```

```

"subscriptions": [
  {
    "id": "s1",
    "followerId": "u1",
    "followingId": "u2"
  },
  {
    "id": "s2",
    "followerId": "u1",
    "followingId": "u3"
  },
  {
    "id": "s3",
    "followerId": "u2",
    "followingId": "u1"
  },
  {
    "id": "s4",
    "followerId": "u3",
    "followingId": "u1"
  }
],
"discussions": [
  {
    "id": "c1",
    "itemType": "model",
    "itemId": "m1",
    "author": "ml_user42",
    "text": "Очень удобная модель для быстрого прототипирования и анализа отзывов."
  },
  {
    "id": "c2",
    "itemType": "model",
    "itemId": "m1",
    "author": "data_researcher",
    "text": "Хотелось бы увидеть сравнение с RoBERTa на этом же датасете."
  },
]

```

Сервер же запускается через команду:

```
npx json-server-auth db.json --port 3000
```

2. Регистрация авторизации

Для регистрации используется POST-запрос:

```

try {
  await axios.post(API_URL + '/register', payload);

  const modal = new bootstrap.Modal(document.getElementById('registerSuccessModal'));
  modal.show();
  event.target.reset();
} catch (error) {
  console.error(error);
  alert('Не удалось подключиться к серверу:');
}

```

Для авторизации:

Авторизация пользователя реализована через отправку POST-запроса на эндпоинт /login с использованием Axios. После успешного ответа токен и данные пользователя сохраняются в localStorage

```
try {
  const response = await axios.post(API_URL + '/login', { email, password });
  const user = response.data.user || {};

  localStorage.setItem('accessToken', response.data.accessToken);
  localStorage.setItem('userId', user.id || '');
  localStorage.setItem('userName', user.name || '');
  localStorage.setItem('userEmail', user.email || '');
  localStorage.setItem('userRole', user.role || '');

  window.location.href = 'profile.html';
} catch (error) {
  console.error(error);
  alert('Не удалось войти:(');
}
```

3. Хранение данных пользователя

После входа данные сохраняются в localStorage

4. Работа с API

Получение данных:

```
async function loadProfile() { Show usages
  requireAuth();
  const userId = localStorage.getItem('userId');
  const token = localStorage.getItem('accessToken');
  const headers = token ? { Authorization: 'Bearer ' + token } : {};

  try {
    const [usersRes, modelsRes, datasetsRes, subscriptionsRes] = await Promise.all([
      axios.get(API_URL + '/users', { headers }),
      axios.get(API_URL + '/models'),
      axios.get(API_URL + '/datasets'),
      axios.get(API_URL + '/subscriptions')
    ]);

    const users = usersRes.data;
    const models = modelsRes.data;
    const datasets = datasetsRes.data;
    const subscriptions = subscriptionsRes.data;

    usersMap = {};
    users.forEach(user => usersMap[user.id] = user);
    const currentUser = usersMap[userId];
```

Для отображения данных профиля использовались GET-запросы к моковому API

Отправка данных:

```
try {
  await axios.post(API_URL + '/discussions', {
    itemType: currentItem.type,
    itemId: currentItem.id,
    author: userName,
    text
  });

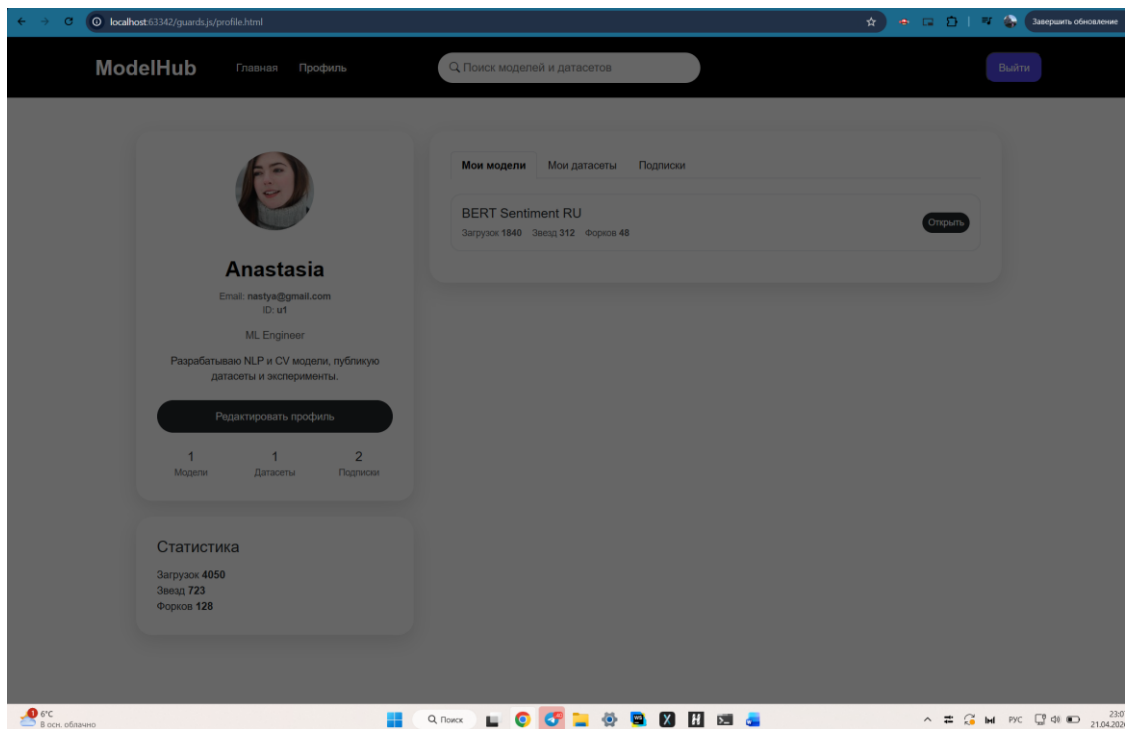
  textarea.value = '';
  loadItem();
} catch (error) {
  console.error(error);
  alert('Ошибка отправки комментария');
}
```

Добавление комментариев через POST

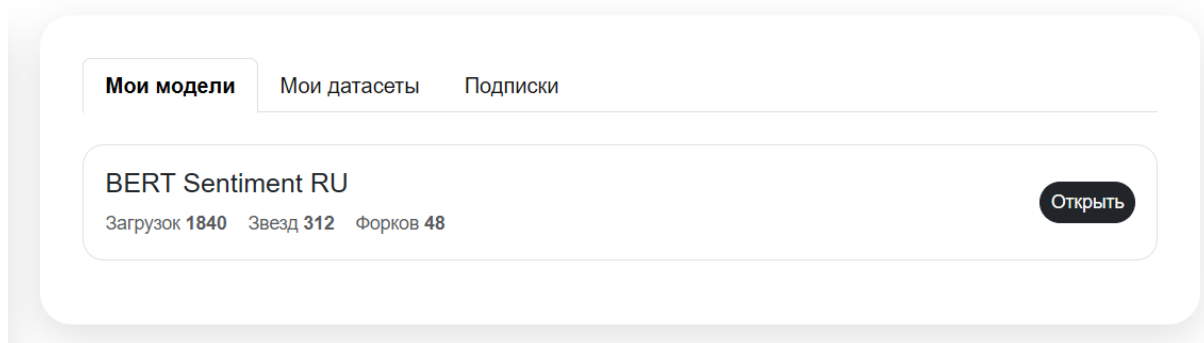
5. Интеграция с интерфейсом

Полученные данные используются для:

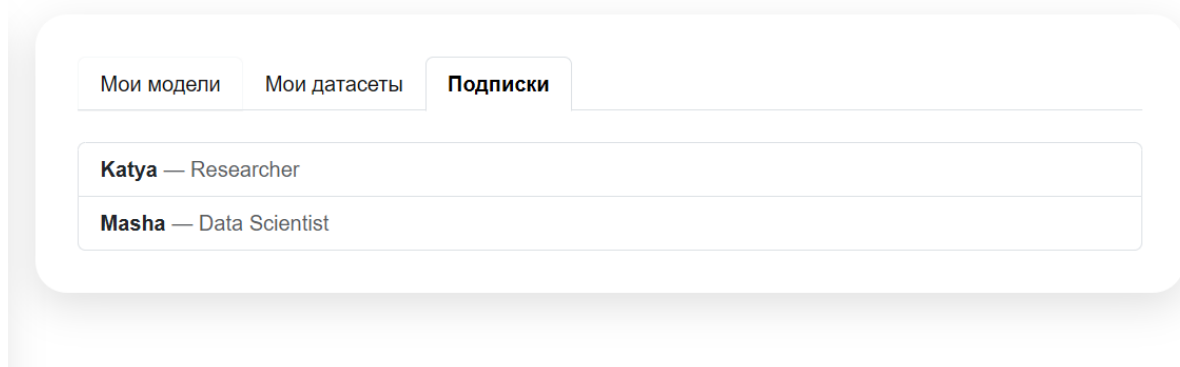
- отображения профиля пользователя



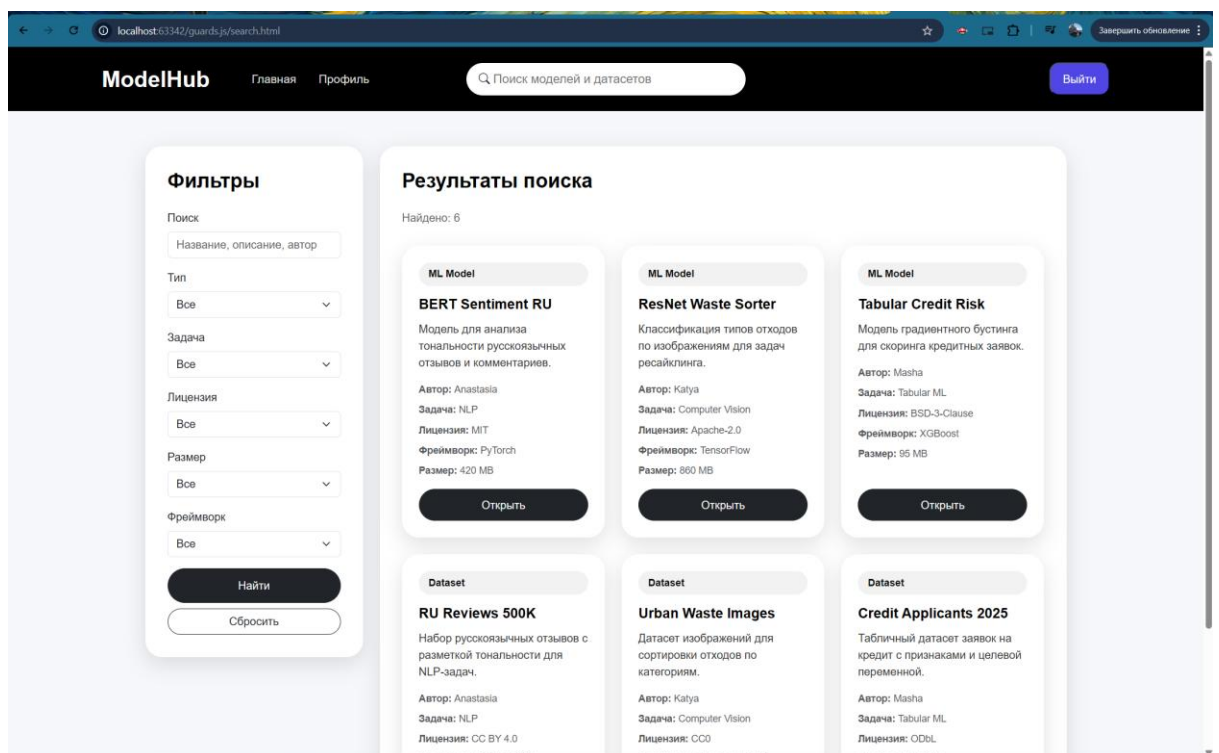
- вывода списка моделей и датасетов



- отображения подписок

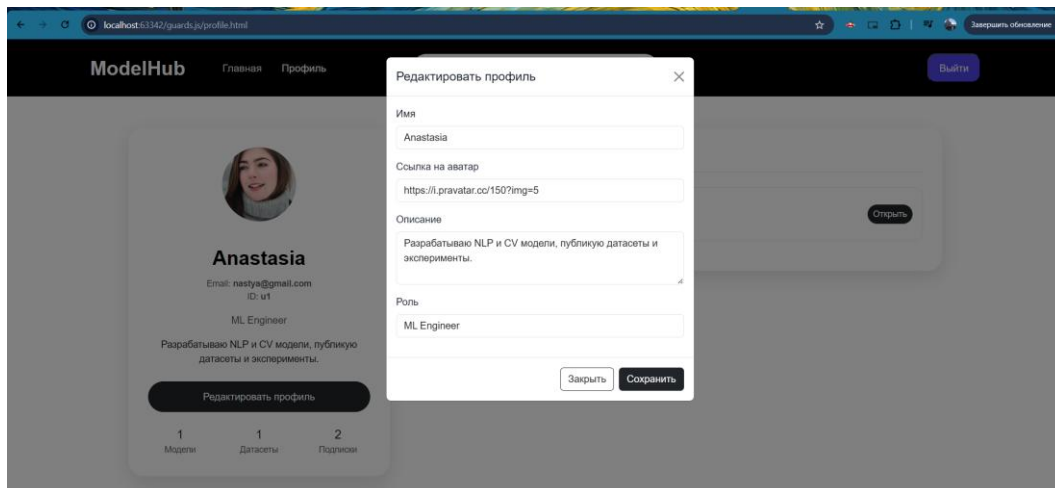


- формирования карточек моделей (название, звезды, форки, загрузки)

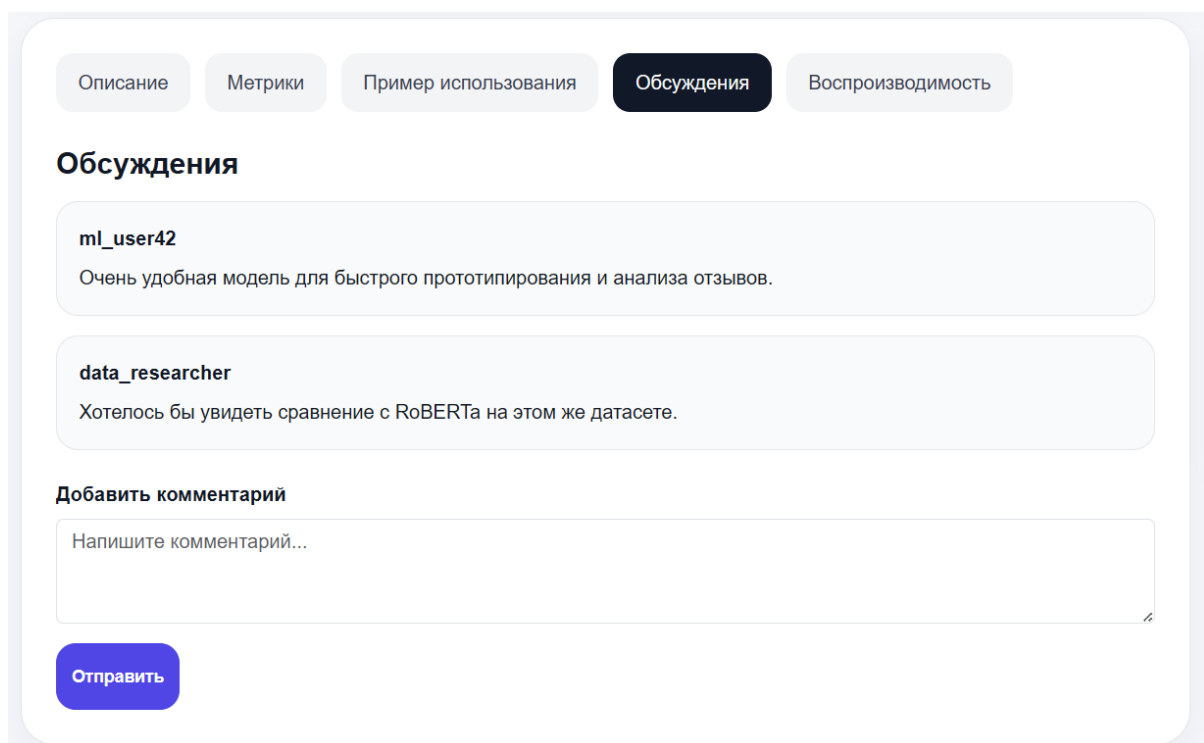


Реализованы функции:

- редактирование профиля



- изменение аватара
- изменение описания
- добавление комментариев



6. Хранение данных

Данные хранятся в db.json, токен в localStorage, пароли хранятся в виде кэша

Вывод

В ходе выполнения лабораторной работы были изучены принципы взаимодействия фронтенда с API. Освоена работа с HTTP-запросами с использованием Axios, реализована авторизация пользователей и хранение данных. Использование JSON Server позволило реализовать серверную часть без разработки полноценного backend, что значительно ускорило процесс разработки и тестирования приложения. Разработанное приложение демонстрирует полный цикл взаимодействия с API: от отправки запросов до отображения данных пользователю.